

GLOBAL ENERGY RISK MONITOR

Technical Architecture Audit & Performance Walkthrough

Document Type:	Internal Technical Architecture & Verification Specification
Target Audience:	Architecture Review Panel & Technical Mentors
Core Engine:	Native C++ Computational Core (GCC/Clang std::c++11)
Bridge Layer:	Asynchronous Node.js Microservice Server (v24.13.0)
AI Integration:	Google Gemini API (gemini-2.0-flash) with Caching & Simulation
Database:	Firebase Realtime Database REST Synchronization Engine
Date Created:	May 25, 2026

Global Energy Risk Monitor — Comprehensive Architecture Audit & Technical Walkthrough

Welcome to the central technical architecture review and engineering walkthrough of the **Global Energy Risk Monitor System**. This document serves as a senior-level technical handover, system walkthrough, and architectural review for senior developers, technical mentors, and architecture review panels.

1. End-to-End System Architecture

The Global Energy Risk Monitor is an advanced, hybrid desktop-web analytics platform that couples a high-performance **C++ computational core** with an asynchronous **Node.js API bridge server** and a **premium, glassmorphic single-page web dashboard**.

All risk modeling, statistical calculations, dataset normalizations, and moving averages are computed directly in native C++. The Node.js server acts as an orchestrator, executing the C++ engine via command-line flags, capturing structured JSON streams from standard output, managing asynchronous external calls (Google Gemini API, Firebase Realtime Database), and serving the web dashboard.

Architectural Diagram

```
graph TB
    subgraph "Data & Storage Layer"
        D1["data/Global Energy Dataset (1900-2024).csv<br/>Kaggle Primary Timeline"]
        D2["data/Global Fuel Prices (2020-2026).csv<br/>Brent Crude & Local Index"]
        D3["data/Gobal Energy Consumption (2000-2024).csv<br/>National Energy Demand"]
        D4["data/energy_data.csv<br/>14 Canonical Resources"]
        D5["data/events.csv<br/>10 Geopolitical Events"]
        FB["Firebase Realtime Database<br/>Cache and Synchronization REST API"]
    end

    subgraph "C++ Backend Computational Core"
        MC["main.cpp<br/>JSON Flag Gateway / Console Menu Loop"]
        DL["data_loader.cpp<br/>CSV stream I/O parser"]
        RE["risk_engine.cpp<br/>5-Factor score aggregator"]
        AN["analysis.cpp<br/>Supply & region sorters"]
        DP["display.cpp<br/>CLI formatting & summary"]
        JA["json_api.cpp<br/>JSON Serialization & Endpoint Router"]
        DI["dataset_importer.cpp<br/>Kaggle CSV Parser & Normalizer"]
        AE["analytics_engine.cpp<br/>Timeline Merger, MA & CAGR Engine"]
    end

    subgraph "Bridge & Orchestration Layer (Node.js)"
        SV["server.js<br/>HTTP Bridge & Request Router"]
        RN["RegionNormalizer.js<br/>Alias Resolving Map"]
        FR["FirebaseRegionInsightRepository.js<br/>Firestore-Style DB REST repository"]
        GF["GeminiFallbackService.js<br/>Prompt Orchestrator & Live Simulation"]
        GV["GeminiResponseValidator.js<br/>Response Envelope Schema Validator"]
        AV["analyticsValidator.js<br/>Centralized Data Validation Layer"]
    end

    subgraph "UI/UX Layer (Vanilla Web Dashboard)"
        HT["index.html<br/>Dynamic Glassmorphism Container"]
        CS["styles.css<br/>1600+ Lines Custom HSL Styling"]
        AP["app.js<br/>Leaflet.js Map, Charts.js & State Manager"]
    end

    %% Data Sync
    FB <-->|"REST HTTPS Sync"| SV
    SV -->|"FileSync"| D4
    SV -->|"FileSync"| D5

    %% C++ execution
    D4 --> DL
    D5 --> DL
    D1 --> DI
    D2 --> DI
    D3 --> DI
    DL --> RE
    DI --> AE
```

```
RE --> JA
AE --> JA
MC --> JA

%% Bridge Interaction
SV -->|"execFile --json"| MC
JA -->|"JSON stdout stream"| SV

%% AI and validation
SV <-->|"REST HTTPS"| GF
GF <-->|"Validates Payload"| GV
GF <-->|"Data Gating"| AV

%% UI interactions
SV -->|"HTTP REST API"| AP
AP --> HT
CS --> HT
```

2. Complete File & Folder Map

```

PROJECT_Energy_Risk_Manager/
|
├─ README.md                ← Base project indicator
├─ compile_flags.txt         ← Clangd configurations
├─ Project_Walkthrough.pdf   ← Visual analysis report
|
└─ EnergyRiskMonitor/
    ├─ .env                  ← Configurations (GEMINI_API_KEY, FIREBASE_DATABASE_URL)
    |
    ├─ data/                 ← DATA STORAGE LAYER (CSV)
    |   ├─ Global_Energy_Dataset_(1900-2024).csv    ← 126-column timeline (Kaggle)
    |   ├─ Global_Fuel_Prices_(2020-2026).csv      ← Brent & retail fuel prices
    |   ├─ Gobar_Energy_Consumption_(2000-2024).csv ← Consumption and price index
    |   ├─ energy_data.csv      ← Active 14 energy resources
    |   ├─ events.csv           ← Active 10 geopolitical threat events
    |   └─ risk_scores_output.csv ← Output calculated risk files
    |
    ├─ backend/              ← C++ COMPUTATIONAL CORE
    |   ├─ CMakeLists.txt     ← Build configuration for GCC/Clang
    |   ├─ include/           ← C++ Header files
    |   |   ├─ data_loader.h   ← Structs for EnergyResource, GeopoliticalEvent
    |   |   ├─ risk_engine.h   ← RiskScore definition
    |   |   ├─ features.h      ← Lookup and event prototypes
    |   |   ├─ analysis.h      ← Disruption & comparison prototypes
    |   |   ├─ display.h       ← Formatting and bar graph prototypes
    |   |   ├─ json_api.h      ← Web CLI interface
    |   |   └─ dataset_importer.h ← Raw Kaggle CSV parser structs
    |   └─ analytics_engine.h ← SMA, CAGR, and trend math definitions
    |
    └─ src/                  ← C++ Source files
        ├─ main.cpp           ← Entry check for --json flag & fallback console loop
        ├─ data_loader.cpp     ← Reads CSV stream line-by-line using stringstream
        ├─ risk_engine.cpp     ← Computes 5 sub-scores & dynamically weights them
        ├─ features.cpp        ← Resource searching & active event filtering
        ├─ analysis.cpp        ← Sorting & regional averaging
        ├─ display.cpp         ← Standard out table drawing & global summaries
        ├─ json_api.cpp        ← Out-stream JSON serializations for Web
        └─ dataset_importer.cpp ← High-efficiency Kaggle CSV column index parser
            └─ analytics_engine.cpp ← Timeline merger, compatible fallback, SMA calculations
    |
    └─ services/             ← BRIDGE & ORCHESTRATION LAYER (Node.js)
        ├─ server.js          ← REST Router, CORS Preflight, CSV synchronizer
        ├─ RegionNormalizer.js ← Maps codes/abbreviations to unique IDs
        ├─ FirebaseRegionInsightRepository.js ← Firebase CRUD REST client
        ├─ GeminiFallbackService.js ← Retry logic, dataset quality checker & local simulation
        └─ GeminiResponseValidator.js ← Strict response envelope validator
  
```

```
|   └─ analyticsValidator.js ← Data quality estimator & Standard Unit Validator
|
└─ frontend/                ← DYNAMIC PRESENTATION LAYER
    └─ index.html           ← Single-page frame, stats card container & modal layouts
    └─ styles.css           ← Custom theme layout with custom scrollbars & particle animations
    └─ app.js               ← Map controller, Charts.js renderer & state sync engine
```

Detailed Component Audits

C++ BACKEND COMPONENTS

Component / File	Purpose	Data Received	Data Returned	Core Logic / Equations
main.cpp	System entry point.	Command line arguments (<code>argc</code> , <code>argv</code>).	<code>0</code> or error status.	Checks for <code>--json</code> flag. Redirects to <code>json_api.cpp</code> if present, else launches terminal CLI loop.
data_loader.cpp	CSV file reader and writer.	Absolute paths to CSVs.	<code>vector<EnergyResource></code> , <code>vector<GeopoliticalEvent></code> .	Parses CSV files line-by-line. Uses <code>stringstream</code> to split tokens by commas and convert numeric columns using <code>stod()</code> and <code>stoi()</code> .
risk_engine.cpp	Mathematical risk calculator.	<code>EnergyResource</code> , <code>vector<GeopoliticalEvent></code> .	<code>RiskScore</code> struct.	Computes 5 distinct risk sub-scores: Supply, Conflict, Trade, Demand, and Route. Aggregates via dynamic weighted average. Clamps results between <code>0.0</code> and <code>100.0</code> .
dataset_importer.cpp	Raw Kaggle dataset validator.	CSV file paths.	Serialized JSON array of normalized timelines.	Parses massive CSV files, extracts specific columns (e.g. coal/oil/gas consumption/production), normalizes country names, and generates quality control metadata.
analytics_engine.cpp	C++ analytical parser.	Country, commodity, min/max years, paths.	<code>AnalyticsResult</code> struct.	Merges datasets. Performs compatible fallback scaling (e.g., Gas = 60% of Oil). Computes 3-year Simple Moving Averages and compound growth rates (CAGR).
json_api.cpp	C++ JSON serializer.	Parsed vectors.	JSON printed to stdout.	Maps commands (dashboard, resources, events, regions, search, etc.) to target logic. Escapes special characters using a custom out-stream buffer.

NODE.JS BRIDGE COMPONENTS

Component / File	Purpose	Data Received	Data Returned	Core Logic / Equations
server.js	HTTP REST Controller & Gateway.	HTTP queries, requests, CSV sync schedules.	Static files, API JSON.	Spawns <code>EnergyRisk.exe --json <args></code> as a child process. Handles CORS and POST payloads. Syncs Firebase collections directly with local CSV datasets before C++ runs.
RegionNormalizer.js	Inputs cleaning.	Raw string country query.	Normalized ID & display name.	Uses <code>REGION_ALIASES</code> to resolve synonyms (e.g., "PRC" → <code>china</code>). Outputs Firebase-safe strings.
FirebaseRegionInsightRepository.js	DB caching repository.	Entity schemas.	Read/Write completions.	Uses standard HTTPS REST endpoints under <code>/gemini_region_insights/{id}.json</code> with a 6-hour Cache TTL.
GeminiFallbackService.js	Gemini orchestrator.	Region name, structured evidence.	Structured JSON report.	Implements a double-retry pattern. Gated by dataset validators. Generates high-quality simulated reports for major countries if Gemini key is invalid.
analyticsValidator.js	Out-of-bounds validator.	Commodity records, years.	Data quality reports.	Validates pricing constraints (e.g., Oil Brent limits \$20-\$200). Generates <code>StructuredEvidenceObjects</code> for AI prompt grounding.

3. Data Flow Analysis

This trace documents the complete lifecycle of a request, starting with a user selecting a country on the interactive world map, continuing through backend calculation and AI validation, and ending with frontend rendering.

```

[User Action]
  ↓
User clicks "Russia" on Leaflet World Map in the web dashboard.
  ↓
[State Update & Routing]
`app.js` triggers `showCountryInsight("Russia")`.
`app.js` calls `fetch("/api/region-insights?region=Russia")`.
  ↓
[Bridge Request Interception]
`server.js` intercepts GET request.
Routes query to `RegionInsightController.js`.
  ↓
[Input Normalization]
`RegionNormalizer.js` parses "Russia".
Returns `{ region_id: "russia", region_name: "Russia", was_alias: true }`.
  ↓
[Firebase Cache Checking]
`FirebaseRegionInsightRepository.js` reads `/gemini_region_insights/russia.json`.
*Case A (Fresh Cache)*: Returns cached JSON. Flow jumps to Rendering.
*Case B (Cache Miss/Expired)*: Proceed to Gating & Validation.
  ↓
[Centralized Data Validation Layer]
`analyticsValidator.js` is triggered.
Loads `datasets/energy_consumption`, `datasets/fuel_prices`, `datasets/global_energy`.
Calculates CAGR, YoY trend variance, and checks for out-of-bounds prices.
Generates an array of `StructuredEvidenceObjects` (Grounding Evidence Block).
  ↓
[AI Context Generation]
`GeminiFallbackService.js` builds prompt containing the Grounding Evidence Block.
Enforces strict grounding requirements (no hypothetical CAGR/price inflation, specific names only).
  ↓
[Gemini API Execution]
Calls the Google Generative AI REST endpoint with `gemini-2.0-flash`.
*Sub-Flow (If Gemini fails / Invalid API key)*:
1. Catches network/authentication error.
2. Checks Firebase cache. If cache exists, returns cached data.
3. If cache is empty, calls `generateSimulatedInsight("Russia")`.
4. Writes the generated report to Firebase cache so subsequent requests hit cache.
  ↓
[Response Schema Validation]
`GeminiResponseValidator.js` validates returned JSON payload.
Injects safe fallback values for any missing or mistyped properties.
  ↓
[Firebase Caching]
Controller writes the validated, complete report back to `/gemini_region_insights/russia.json`.
  ↓
[C++ Computational Engine Invocation]
Meanwhile, frontend fetches C++ resource metrics.
`server.js` executes: `EnergyRisk.exe --json resource OIL_RUS` as a child process.
C++ `main.cpp` detects `--json` flag → calls `json_api.cpp` → loads CSVs → calculates risk dynamically.

```

```
C++ writes JSON to `stdout` stream. Node.js captures stdout and returns it to the client.
```

↓

```
[Frontend Shaping & Render]
```

```
`app.js` receives both C++ resource metrics and Gemini/Simulated Insights.
```

```
Charts.js instance destroys old canvases and shapes new datasets.
```

```
Renders Glassmorphic insight cards, Leaflet map colors, and animates stats.
```

4. Analytics Engine Explanation

The C++ Analytics Engine (`analytics_engine.cpp`) is responsible for compiling raw, high-density historic timelines across three distinct datasets:

1. **Global Energy Dataset (1900-2024)** (126-column timeline)
2. **Global Fuel Prices (2020-2026)** (Brent index and local fuel pricing)
3. **Global Energy Consumption (2000-2024)** (National electricity and energy demand indices)

Key Architectural Mechanisms

1. TARGET COMMODITY MAPPING

Since raw datasets contain variations in terminology, the engine normalizes requested commodities (e.g. Natural Gas, LNG) to standard types (Oil, Gas, Coal, Electricity, Renewables, Nuclear) using `DataNormalizer::normalizeEnergyType()` .

2. COMPATIBLE FALLBACK SCALING HEURISTICS

If a country is completely missing a targeted commodity series, the engine performs compatible fallback scaling using a proportional scale factor against available commodities:

- **Gas/LNG missing:** Scaled from available `oil` metrics with a factor of `0.6` (Gas is ~60% of Oil's energy scale in equivalent units).
- **Coal missing:** Scaled from `gas` with a factor of `0.8` (Coal is ~80% of Gas scale).
- **Renewables missing:** Scaled from `Electricity` with a factor of `0.15` (Renewables make up ~15% of the average electricity generation mix).
- **Nuclear missing:** Scaled from `Electricity` with a factor of `0.10` (~10% mix).
- **Electricity missing:** Scaled from `gas` with a factor of `0.5` (~50% gas scale).

3. SELF-SUFFICIENCY ESTIMATION

If production data is missing but consumption is present, the engine estimates production:

- For known net exporters (Saudi Arabia, Russia, Qatar, USA), production is estimated at `1.30 * consumption` .
- For others, a baseline import-dependent self-sufficiency factor of `0.95 * consumption` is applied.

4. PRICING FALLBACK STRATEGY

Direct retail prices (Petrol, Diesel, LPG) are looked up first. If missing, price series are extracted from consumption index columns. If both are missing, a **Scaled National Index** is generated:

- The engine takes the global Brent crude oil price and applies commodity-specific multipliers:
$$\text{Coal Price} = \text{Oil Price} \times 80.0$$
$$\text{Electricity Price} = \text{Oil Price} \times 0.15$$
$$\text{Renewables Price} = \text{Oil Price} \times 0.06$$
$$\text{Gas Price} = \text{Oil Price} \times 5.0$$

5. MOVING AVERAGE (SMA) CALCULATIONS

A 3-year Simple Moving Average is calculated dynamically for each year t :

$$\text{SMA}_t = \frac{\text{Value}_t + \text{Value}_{t-1} + \text{Value}_{t-2}}{\text{Count of Valid Years}}$$

This effectively smooths out outliers and represents long-term demand changes.

6. COMPOUND ANNUAL GROWTH RATE (CAGR)

The consumption CAGR is calculated over the first valid year t_{start} and latest year t_{end} :

$$\text{CAGR} = \left(\frac{\text{Value}_{t_{\text{end}}}}{\text{Value}_{t_{\text{start}}}} \right)^{\frac{1}{t_{\text{end}} - t_{\text{start}}}} - 1.0$$

This value is converted to a percentage in JSON serialization.

5. Risk Engine Explanation

The risk engine (`risk_engine.cpp`) is a 5-factor mathematical model that quantifies risks on a scale of `0.0` to `100.0` . The final risk level classification is defined as:

- **HIGH:** Score > 66.0
- **MEDIUM:** Score > 33.0
- **LOW:** Score ≤ 33.0

The 5 Sub-Score Formulas

FACTOR 1: SUPPLY RISK SCORE ($SS_{\{SUPPLY\}}$) — WEIGHT: 25%

Measures the balance between consumption (C) and production (P). A ratio ($R = C / P$) is computed:

- If $P \leq 0$, supply risk is maximum: $SS_{\{supply\}} = 100.0$.
- If $R < 1.0$ (production exceeds consumption), the risk scales linearly:

$$SS_{\{supply\}} = R \times 40.0$$
- If $R \geq 1.0$ (consumption exceeds production), risk escalates:

$$SS_{\{supply\}} = 40.0 + (R - 1.0) \times 30.0$$
- Clamped between 0.0 and 100.0 .

FACTOR 2: CONFLICT RISK SCORE ($SS_{\{CONFLICT\}}$) — WEIGHT: 30%

Measures regional military and political instability. Derived from the set of active events ($E_{\{active\}}$) in the resource's region where event type is **War** or **Instability**:

- If the resource has no export dependency ($D_{\{export\}} \leq 0.0$), it is insulated: $SS_{\{conflict\}} = 10.0$.
- If no active conflict events exist in the region, a default baseline is used:

$$SS_{\{conflict\}} = 15.0 \times D_{\{export\}}$$
- If active events exist, the score is calculated using average event intensity ($I_{\{avg\}}$):

$$SS_{\{conflict\}} = 15.0 + I_{\{avg\}} \times 60.0 \times D_{\{export\}}$$
- Clamped between 0.0 and 100.0 .

FACTOR 3: TRADE RISK SCORE ($SS_{\{TRADE\}}$) — WEIGHT: 20%

Measures exposure to sanctions, tariffs, and trade restrictions. Derived from active events in the region where event type is **Sanctions** or **TradeRestriction**:

- Base trade risk is calculated from export dependency: $SS_{\{base\}} = D_{\{export\}} \times 15.0$.
- For each active trade restriction, the raw intensity percentage is added directly:

$$SS_{\{trade\}} = S_{\{base\}} + \sum (I_{\{event\}} \times 100.0)$$
- Capped at 98.0 (representing that trade can rarely be completely severed).

FACTOR 4: DEMAND RISK SCORE ($SS_{\{DEMAND\}}$) — WEIGHT: 15%

Measures exposure to depleting national reserves and global export dependency:

- Base demand risk: $SS_{\{base\}} = 15.0 + D_{\{export\}} \times 35.0$.

- If remaining years of reserves (Y_{reserves}) is less than 60 years, a penalty is added:

$$S_{\text{penalty}} = \left(\frac{60.0 - Y_{\text{reserves}}}{60.0} \right) \times 45.0$$
- Clamped between \$0.0\$ and \$100.0\$.

FACTOR 5: ROUTE RISK SCORE (S_{ROUTE}) — WEIGHT: 10%

Measures geographical vulnerability of trade corridors and shipping chokepoints. Uses a regional lookup table to establish baseline route risk (R_{route}):

- Middle East / Saudi Arabia: \$65.0\$ (Strait of Hormuz, Bab-el-Mandeb chokepoint exposure)
- Russia: \$48.0\$ (Subsea pipelines and Baltic bottlenecks)
- China: \$40.0\$ (Strait of Malacca strategic exposure)
- Venezuela: \$35.0\$ (Panama Canal and Caribbean shipping lanes)
- European Union: \$22.0\$ (North Sea pipelines)
- USA: \$15.0\$ (Gulf Coast storm risks)
- All other regions: \$20.0\$ (Standard baseline)

The final score scales the baseline risk by the export dependency:

$$S_{\text{route}} = R_{\text{route}} \times (0.4 + 0.6 \times D_{\text{export}})$$

Dynamic Weighted Synthesis

To handle sparse or missing data, the engine dynamically excludes components with negative sentinel values (e.g. $S_{\text{sub}} < 0.0$):

$$\text{Composite Score} = \frac{\sum (S_{\text{sub}} \times w_{\text{sub}})}{\sum w_{\text{sub}}}$$

If fewer than 3 valid sub-scores exist, the engine classifies the record as **INSUFFICIENT** and returns a sentinel score of **-1.0** to prevent misleading risk projections.

6. Gemini Integration Explanation

The platform features an advanced integration with the Google Gemini API (`gemini-2.0-flash`) via the REST interface.



The Grounding Constraint Prompt

Prompt engineering is used to anchor Gemini's generation in validated local data:

- **Evidence-Based Grounding:** The prompt injects a raw JSON array of verified data points (consumption, prices, historical ranges, and sources) and forbids Gemini from generating any statistics, CAGR, or prices not present in this block.
- **Refining Specificity:** Standard generative AI tend to produce generic phrases (e.g. "ongoing geopolitical tensions"). The prompt implements **Strictest Specificity Constraints**:
 1. *Specific Infrastructure:* Requires naming actual facilities (e.g. "Ras Tanura refinery", "Nord Stream pipeline") instead of generic "energy assets".
 2. *Specific Conditions:* Requires naming the exact geopolitical entities (e.g. "OPEC production quotas") rather than "market pressures".

3. *Specific Commodities*: Requires specifying the precise fuel types (e.g. "thermal coal", "Qatari LNG") instead of "resources".
4. *Locality Enforcement*: Forbids the response from containing sentences that could describe other countries without changing the words. If a finding is not region-locked, it is omitted.

High-Quality Local Simulation

If the Gemini API key is missing, expired, or rate-limited, the system falls back to a **high-quality local simulation model** in `GeminiFallbackService.js`.

This simulator contains curated, region-locked risk profiles for major global energy regions (Russia, Saudi Arabia, Germany/EU, USA, China, Ukraine, Norway) and a standard fallback model. These simulated models are structured to match the expected JSON schema:

```
{
  "region_name": "Norway",
  "query_text": "Geopolitical energy risk analysis for Norway",
  "generated_summary": "Norway is Europe's largest supplier of natural gas...",
  "energy_risks": "Hydropower reservoirs are vulnerable to seasonal precipitation...",
  "trade_supply_chain_risks": "High dependency on submarine pipeline networks...",
  "infrastructure_risks": "Subsea pipelines require heightened security due to threats of undersea sabotage...",
  "affected_resources": ["gas", "oil", "electricity", "renewables"],
  "geopolitical_events": [
    {
      "title": "Undersea Pipeline Sabotage Threat",
      "description": "Increased naval patrols...",
      "severity": "high",
      "affected_resources": ["gas"],
      "region_impact": "Heightened security costs..."
    }
  ],
  "supply_risk_level": "low",
  "fuel_price_impact": {
    "level": "medium",
    "summary": "High gas export prices support state revenues..."
  },
  "import_export_vulnerabilities": [
    "Heavy reliance on subsea pipelines to Germany and UK...",
    "Exposure of wind and hydro sectors to seasonal climate..."
  ],
  "recommendation": "Enhance drone detection around offshore platforms..."
}
```

Once a simulated report is generated, the controller writes it to the Firebase Realtime Database. Subsequent queries for that country are loaded from the cache, preventing redundant simulations.

7. Firebase & Data Storage

The platform uses the **Firebase Realtime Database** via its HTTPS REST API.

Database Schema Structure

```

firebase-database-root/
|
├─ resources/                ← Realtime resource definitions
|   └─ 0/                    ← Array index
|       └─ id: "OIL_SAU"
|       └─ name: "Saudi Crude Oil"
|       └─ ... (production, consumption, price)
|   └─ 13/
|
├─ events/                   ← Dynamic geopolitical events array
|   └─ 0/
|       └─ id: "E001"
|       └─ title: "Russia-Ukraine War"
|       └─ is_active: 1
|   └─ 9/
|
├─ submitted_events/         ← User-submitted crowd-sourced events
|   └─ {20_char_auto_id}/
|       └─ id: "abcXYZ123..."
|       └─ title: "Refined Fuel Pipeline Freeze"
|       └─ severity: "critical"
|       └─ description: "Severe cold front freezing distribution pipelines..."
|
├─ country_risk_scores/      ← Aggregated map risk metrics
|   └─ SAU/
|       └─ country_code: "SAU"
|       └─ risk_score: 28.5
|       └─ risk_level: "low"
|   └─ RUS/
|
├─ gemini_region_insights/   ← Gemini AI report cache
|   └─ {normalized_region_id}/
|       └─ region_name: "Russia"
|       └─ generated_summary: "..."
|       └─ updated_at: "2026-05-24T22:00:00.000Z"
|       └─ ... (validated schema payload)
|
└─ datasets/                 ← Primary Kaggle historical series
    └─ energy_consumption/
    └─ fuel_prices/
    └─ global_energy/
  
```

Data Synchronization Flow

- At startup, `server.js` checks if the `resources` and `events` endpoints exist in Firebase.
 - If they are empty, it seeds the database with canonical defaults.
 - Before executing any C++ backend operations, `server.js` calls `ensureDataSync()`. This reads the latest state from Firebase and overwrites `data/energy_data.csv` and `data/events.csv` locally.
 - This ensures that any runtime additions (such as crowd-sourced geopolitical events or registered energy resources) are available to the C++ parser.
-

8. UI/UX System Explanation

The frontend is a vanilla JavaScript single-page application built on CSS Grid and Flexbox layouts.



Interactive Features

1. LEAFLET CHLOROPLETH MAP

Loads global country borders via GeoJSON and overlays color-coded risk layers (Green = Low, Amber = Medium, Red = High, Deep Burgundy = Critical). Hovering over a country displays a Leaflet custom popup showing active resources and events. Clicking a country focuses the map and loads regional insights.

2. DEBOUNCED SEARCH GATEWAY

The search input features a 250ms debounce mechanism:

```
let debounceTimer;
input.addEventListener('input', (e) => {
  clearTimeout(debounceTimer);
  debounceTimer = setTimeout(() => triggerSearch(e.target.value), 250);
});
```

This reduces server load by preventing rapid API requests for every keystroke.

3. MODAL DETAILS CONTROLLER

Clicking any resource in the dashboard tables triggers an asynchronous fetch to `/api/resource/{id}`. This queries the C++ backend for the resource's calculated sub-scores and queries Firebase for related events, displaying them in a modal overlay.

9. Known Issues & Architectural Limitations

1. Child Process Execution Overhead:

Using `execFile` for every API query spawns a new OS process (`EnergyRisk.exe`). For high-traffic web applications, this approach is highly inefficient, leading to high CPU usage and response latency.

2. Hardcoded C++ Route Risk:

`risk_engine.cpp` contains a hardcoded `calcRouteScore` lookup table (e.g. Middle East = 65, USA = 15). This limits flexibility, as shipping chokepoint risks cannot be updated dynamically without recompiling the binary.

3. Sparse Data Gating in Fallbacks:

When Kaggle CSV data is missing, the C++ engine relies on basic fallback scaling factors (e.g. Gas = 60% of Oil). While useful for preventing empty charts, these estimates are static and do not reflect real-world energy dynamics.

4. Node.js Single-Threaded CSV Parsing Fallback:

If the C++ importer fails, the Node.js server reads the massive CSV files synchronously into memory. This blocks the main event loop, causing temporary API freezing for other users.

5. No State Persistence on Server Interruption:

The Node.js server stores transient country indices in memory. If the server restarts without a configured Firebase database, active user-submitted events are lost.

10. Technical Improvement Roadmap

SHORT-TERM (1-2 Weeks)	MID-TERM (1-2 Months)	LONG-TERM (3-6 Months)
Bindings & Caching - Move process spawning to node-gyp C++ bindings. - Implement Redis memory cache.	Dynamic Risk Routing - Move Route Score lookups to a dynamic JSON file. - Implement real-time shipping data APIs (e.g. AIS).	Serverless C++ Core - Compile C++ core to WebAssembly (Wasm). - Deploy serverless computational lambda functions.

Proposed Action Items

1. Compile C++ as a Shared Library / WebAssembly:

Replace the slow `child_process.execFile` execution bridge with **WebAssembly (Wasm)** or compile the C++ core as a shared library (`.dll` or `.so`) with Node.js N-API bindings. This allows Node.js to call C++ calculations directly in-process, reducing latency to sub-millisecond ranges.

2. Externalize Risk Weighting and Route Tables:

Refactor `risk_engine.cpp` to load the regional route baseline risks and weighting configurations from a configuration file (`config/risk_rules.json`) at startup, allowing developers to fine-tune calculations without recompiling.

3. Integrate Real-Time Maritime Shipping APIs:

Connect the route risk calculation to real-time maritime tracking APIs (e.g., AIS transponder aggregators) to dynamically increase route scores when congestions or threat alerts occur near transit chokepoints.

4. Replace Synchronous Fallbacks with Streams:

Refactor Node.js CSV fallbacks to use stream pipelines (`fs.createReadStream` piped through a transform stream), ensuring that large file parsing never blocks the main thread.

5. Enhance AI Grounding with Vector Embeddings (RAG):

Replace standard prompt grounding with a **Retrieval-Augmented Generation (RAG)** pipeline. Store historical datasets in a vector database, allowing the AI to retrieve relevant data points dynamically rather than injecting large JSON arrays into prompts.